
Deep Learning for Time Series LSST Classification

Charles Karrer^{* 1} Ralph Mouawad^{* 1} Antoine Verier^{* 1}

Abstract

The goal of this project is to classify astronomical time-series data. Large Synoptic Survey Telescope (LSST) is an american astronomical observatory located in Chile which aims to better understand the changing sky, discover and measure millions of objects. In this report, we will first describe the dataset, our approach and then provides several experiments providing classification of these astronomical sources. You could find our pipeline on https://github.com/antoineverier/lsst_classification

1. Exploratory data analysis

1.1. Dataset

The dataset used in this project originates from the Photometric LSST Astronomical Time-series Classification Challenge (PLAsTiCC). It consists of simulated light curves that measure the brightness of celestial objects as a function of time. These measurements are captured across six different astronomical filters, commonly referred to as passbands (u, g, r, i, z, y). As shown during our data exploration using the **tslearn** library, the training subset contains 2459 samples while the test subset contains 2466. Each sample is formatted as a multivariate time series with 36 time steps and 6 distinct features corresponding to the passbands. This dataset contains 14 class. (The PLAsTiCC team et al., 2018)

1.2. Objective

To achieve our classification goals, we chose to tackle the **first setting: Adapt a foundation model**. Our objective is

^{*}Equal contribution ¹École Polytechnique, Institut Polytechnique de Paris, France. Correspondence to: Charles Karrer <charles.karrer@polytechnique.edu>, Ralph Mouawad <ralph.mouawad@polytechnique.edu>, Antoine Verier <antoine.verier@polytechnique.edu>.

to establish a robust, non-pre-trained baseline and compare it against different adapted foundation models. We will explore both simple and complex fine-tuning strategies on foundation models known for their proven adaptability and strong classification capabilities.

2. Methodology

2.1. Evaluation

To effectively assess our models, we must account for the highly skewed class distribution of the PLAsTiCC dataset (as illustrated by the class support distribution in Figure 1.). Because we lack domain-specific priors regarding the relative astronomical importance of each specific event type, we treat all 14 classes as equally important. Consequently, we rely on the Macro F1 score as our primary classification metric. Unlike micro-averaging, the Macro F1 score calculates the metric independently for each class before taking the unweighted mean. This ensures that the performance on minority classes is explicitly preserved and not overshadowed by the dominant classes.

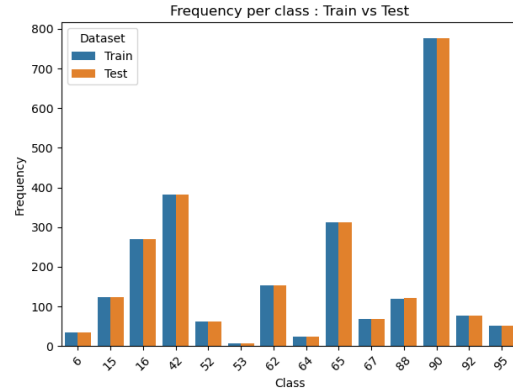


Figure 1. Unbalanced LSST dataset

In addition to the Macro F1 score, we evaluate our models using **weighted logarithmic loss (W-LogLoss)** to align our methodology with the original PLAsTiCC benchmark (The PLAsTiCC team et al., 2018). Log loss provides a strict evaluation of the model’s confidence by heavily penalizing incorrect, high-confidence probability predictions.

This ensures a uniform evaluation pipeline, allowing us to fairly compare the predictive uncertainty of standard linear models, tree-based ensembles, and adapted foundation models.

2.2. Baseline models

To establish a robust benchmark prior to evaluating foundation models, we implement three classical architectures for multivariate time-series classification: a 1D Convolutional Neural Network (1D-CNN) (Wang et al., 2017), a Recurrent Neural Network (RNN), and a Long Short-Term Memory (LSTM) network (Hochreiter & Schmidhuber, 1997).

The **1D-CNN** captures local temporal variations across the 6 passbands. It consists of two convolutional blocks (1D convolution, batch normalization, ReLU, and dropout), followed by max pooling and adaptive average pooling to extract fixed-length representations for a linear classifier. We vary channel dimensions (64, 128) and kernel sizes (3, 5, 7) to evaluate both short- and long-term dependencies.

For sequential processing, we use bidirectional **RNN** and **LSTM** architectures to capture the complete structural profile of transient events by analyzing light curves in both temporal directions. Both networks feature two hidden layers (128 or 256 dimensions), dropout, and a linear classification head.

To address the severe class imbalance, we apply a balanced class weighting scheme to the Cross-Entropy loss. This heavily penalizes misclassifications of rare events (e.g., Classes 53 and 64), preventing the model from defaulting to dominant classes. All models are trained using the Adam optimizer with 50 epochs (learning rate 10^{-3} , L_2 weight decay 10^{-4}). We employ early stopping on the validation loss (patience of 15 epochs) to prevent overfitting and restore optimal weights for final testing.

2.3. First Foundation Models used

2.3.1. MANTIS

Mantis is a time-series foundation model that encodes a univariate input $x \in \mathbb{R}^t$ into an embedding $z = F(x) \in \mathbb{R}^q$. This part includes normalization and difference for generating tokens followed by a vision transformer layer. A classification head $h : \mathbb{R}^q \rightarrow \mathbb{R}^K$ is then added for downstream prediction, either by training only the head (linear probing) or by fine-tuning the whole model. For multivariate series $x = [x_1, \dots, x_d]$, each channel is encoded independently and the final representation is obtained by concatenation, $z = \text{concat}([F(x_i)]_{1 \leq i \leq d})$. In our experiments, we concatenated channel embeddings and evaluated both linear probing and fine-tuning for only 20 epochs to limit overfitting. We applied linear interpolation to make the sequence

lengths to 512 as required by the model’s architecture.

2.3.2. MULTIROCKET

MultiRocket is a fast and efficient time series classification method based on random convolutional kernels. Rather than learning filters through backpropagation, MultiRocket generates a large number of random convolutional kernels of varying lengths and dilations, applies them to the input series, and extracts two simple statistics per kernel: the proportion of positive values (PPV) and the maximum value. This produces a high-dimensional feature vector (up to $\sim 50,000$ features with 6250 kernels) which is then passed to a classifier.

Its main strengths are its speed and its ability to capture diverse temporal patterns through the diversity of random kernels, without requiring any pretraining or GPU resources.

For the training pipeline, we pair MultiRocket head with a logistic regression classifier, based on prior empirical tests. We explore different numbers of kernels ($n_{\text{kernels}} \in \{1000, 6250\}$) and regularization strengths ($C \in \{0.001, 0.01, 0.1, 1, 10, 100\}$).

2.4. MOMENT Foundation Model

The previous foundation models are limited to fine-tuning only the last layer. Given the observed results, we sought to go further by exploring a model allowing deeper fine-tuning. Our research led us to MOMENT-FM (<https://huggingface.co/AutonLab/MOMENT-1-large>), a family of foundation models based on 341M parameters and 24 transformer blocks.

2.4.1. ARCHITECTURAL MODIFICATIONS

By default, MOMENT uses a simple Linear as its classification head. We replaced it with a two-layer MLP: LayerNorm $\rightarrow$ Linear(1024, 256) $\rightarrow$ GELU $\rightarrow$ Dropout(0.5) $\rightarrow$ Linear(256, 14). The idea is to give the model more capacity to adapt the pretrained representations to the specifics of LSST’s 14 classes. Regarding the backbone, we chose to unfreeze the last 4 transformer blocks out of 24. This choice reflects a trade-off: with only 1967 training samples, unfreezing too many layers would inevitably lead to overfitting. The last layers being the most task-specific to pretraining, they benefit the most from adaptation to LSST.

An important technical constraint arose: MOMENT requires sequence lengths to be a multiple of patch_size=8, while LSST has T=36 timesteps. We therefore applied zero-padding to reach T=40, with an attention mask (1 for real timesteps, 0 for padding) passed to the model to ignore artificial positions.

2.4.2. TRAINING STRATEGY

Training proceeds in two phases. First, a warmup phase (15 epochs) where only the new head is trained, with the backbone frozen. This stabilizes the head before touching the pretrained weights. Second, a fine-tuning phase where the last 4 blocks are unfrozen with a differential learning rate: HEAD_LR= 3×10^{-4} for the head and BACKBONE_LR= 5×10^{-5} for the backbone. A cosine annealing scheduler is applied over both phases to refine convergence at the end of training.

3. Experimental Setup

3.1. Data preparation

We use the predefined train/test split from the UCR/UEA repository. The training set is further divided into train and validation using a stratified 80/20 split (1967/492 samples), ensuring class proportions are preserved across splits.

All models share the same preprocessing pipeline: a global standardization is applied by fitting a single StandardScaler on all training values flattened across samples, channels and timesteps — yielding one global mean and one global standard deviation. This scalar normalization is then applied to validation and test sets to prevent any data leakage.

3.2. Implementation details

For all models, hyperparameter selection is based on validation Macro F1, and the best configuration is retrained on the combined train+validation set before final test evaluation. All experiments are run on CPU.

4. Results and discussion

Model	Configuration
1D-CNN	$n_h = 128$ & $kernel_{size} = 5$ $do = 0.3$
1D-RNN	$n_h = 128$ & $n_{layers} = 1$ bidirectional = False & $do = 0.1$
LSTM	$n_h = 128$ & $n_{layers} = 2$ bidirectional = True & $do = 0.3$
Mantis V1	(LP) $C = 10$
Mantis V2	(LP) $C = 1$
MultiRocket	$n_{kernels} = 1000$ & (LP) $C = 0.01$
MultiRocket	$n_{kernels} = 6250$ & (LP) $C = 0.01$
MOMENT-FM	$n_{unfreeze} = 4$ & $do = 0.5$ $n_h = 256$ & $LR_{head} = 3 \times 10^{-4}$ $LR_{backbone} = 5 \times 10^{-5}$

Table 1. Performance (Macro F1 and WLogLoss) of models on test set.

Model	Macro F1 $\uparrow$	W-LogLoss $\downarrow$
1D-CNN	0.3014	2.0490
1D-RNN	0.1788	3.9173
LSTM	0.4205	1.3965
Mantis V1 + LP	0.4672	3.1756
Mantis V2 + LP	0.4778	1.9473
MultiRocket:1000 + LP	0.5095	1.6681
MultiRocket:6250 + LP	0.5278	1.9026
MOMENT-FM	0.4043	3.9449

The severe class imbalance of the 14 astronomical classes made the macro F1 score the primary indicator of generalized performance while W-LogLoss provided insight into model calibration and confidence. Among traditional deep learning baselines, the bidirectional LSTM was by far the most effective. 1D-RNN struggled significantly in every setting indicating poor minority class recognition and overconfident incorrect predictions.

The application of Linear Probing to the Mantis foundation models demonstrated the advantage of leveraging pre-trained representations.

- Mantis V1 achieved a better Macro F1 outperforming all baselines. However, its high W-LogLoss suggests that while it categorizes classes reasonably well, it suffers from overconfident misclassifications. Mantis V2 showed notable improvements in macro F1 score. It slightly increased the Macro F1 to 0.4778 while drastically reducing the W-LogLoss to 1.9473 (using a regularization parameter of $C=1$), indicating a much better-calibrated feature space
- MultiRocket achieved best results overall with quite low log loss and a higher macro F1 score.

Regarding MOMENT-FM, the results fall short of MultiRocket. Several factors explain this. First, MOMENT was pretrained on long sequences, and LSST with $T=36$ provides only 5 patches per sequence, which is very few for a transformer. Second, 1967 samples remain limited for adapting a 341M-parameter model, even partially. These results illustrate a known limitation of time-series foundation models: on short, small-scale datasets, well-optimized classical methods such as MultiRocket remain hard to beat.

References

Feofanov, V., Wen, S., Alonso, M., Ilbert, R., Guo, H., Tiomoko, M., Pan, L., Zhang, J., and Redko, I.

Mantis: Lightweight calibrated foundation model for user-friendly time series classification. *arXiv preprint arXiv:2502.15637*, 2025.

Goswami, M., Saha, A., Dutta, S., et al. Moment: A family of open time-series foundation models. *arXiv preprint arXiv:2402.03885*, 2024. URL <https://arxiv.org/abs/2402.03885>.

Hochreiter, S. and Schmidhuber, J. Long short-term memory. *Neural Computation*, 9(8):1735–1780, 1997.

Tan, C. W., Dempster, A., Bergmeir, C., and Webb, G. I. Multirocket: Multiple pooling operators and transformations for fast and effective time series classification. *arXiv preprint arXiv:2102.00457*, 2021. URL <https://arxiv.org/abs/2102.00457>.

The PLAsTiCC team, Allam Jr., T., Bahmanyar, A., Biswas, R., Dai, M., Galbany, L., Hložek, R., et al. The photometric lsst astronomical time-series classification challenge (plasticc): Data set. *arXiv preprint arXiv:1810.00001*, 2018. URL <https://arxiv.org/pdf/1810.00001>.

Wang, Z., Yan, W., and Oates, T. Time series classification from scratch with deep neural networks: A strong baseline. In *2017 International Joint Conference on Neural Networks (IJCNN)*, pp. 1578–1585. IEEE, 2017.